WorkTrace — Technical Reference

A component-by-component breakdown of how WorkTrace works under the hood.

Table of Contents

---

1. Service topology

Local development

Service	Runtime	Port	Role
PostgreSQL + pgvector	Docker (`docker-compose.yml`)	5433	Document chunks, 768-dim vectors, metadata
Ollama	Native	11434	Local embedding (`nomic-embed-text`) + local LLM (`llama3.2`)
FastAPI / uvicorn	Python	8000	REST API — all backend logic
Vite / React	Node	5173	Frontend SPA

OpenShift (production)

```
Browser → OpenShift Route (HTTPS)
        → frontend pod (nginx :8080)
          → serves static Vite build
          → proxies /api/* → backend ClusterIP :8000
                           → FastAPI pod
                             → in-cluster pgvector (Ceph RBD PVC, port
5432)
                             → Supabase (auth only, external)
                             → Nomic Atlas (embeddings, external)
                             → watsonx / Groq (LLM, external)
```

The backend is never publicly exposed — only the frontend Route is. All API calls go through nginx inside the cluster. `VITE_API_URL=/api` is baked into the frontend image at build time so every fetch goes to the relative `/api` path, which nginx proxies to the backend ClusterIP service.

The separate TTT database (`TTT_DATABASE_URL`) is either the same Supabase/Neon project or a separate one. The main vector database is in-cluster pgvector.

---

2. Data models

`documents` table (pgvector DB)

Defined in `backend/kb/db.py`.

Column	Type	Description
`id`	`INTEGER` PK	Auto-increment
`source`	`VARCHAR(1024)`	Original filename or path — the dedup key
`file_type`	`VARCHAR(64)`	`pdf`, `text`, `docx`, `image`, `generic`
`chunk_index`	`INTEGER`	Position of this chunk within the source (0-based)
`content`	`TEXT`	Raw text of the chunk
`embedding`	`Vector(768)`	pgvector column — cosine similarity target
`created_at`	`TIMESTAMP`	Insert time
`user_id`	`VARCHAR`	Supabase user UUID — all queries are scoped to this
`doc_metadata`	`JSONB`	Parsed meeting header fields: `meeting_date`, `meeting_title`, `organizer`, `attendees`, `platform`, `meeting_time`, `project_code`, `billable`, `duration_minutes`

`doc_metadata` and `user_id` are added via `ALTER TABLE … ADD COLUMN IF NOT EXISTS` in `init_db()` so older deployments without them are migrated transparently on startup. An index on `source` makes dedup checks and source-filter queries fast.

`time_entries` table (TTT DB)

Written to by `backend/kb/pusher.py` and managed by `backend/ttt_api.py`. Read at chat time by `backend/kb/ttt.py`.

Column	Type	Description
`id`	`TEXT` PK	UUID string
`user_id`	`TEXT`	Supabase user UUID — all queries are scoped to this
`project_code`	`TEXT`	Project identifier
`task_type`	`TEXT`	`meeting` for KB-pushed entries; any string for manual entries
`duration_minutes`	`NUMERIC`	Duration in minutes
`entry_date`	`DATE`	Date of the entry
`start_time`	`TIMESTAMPTZ`	Optional start time (naive, no tz conversion)
`end_time`	`TIMESTAMPTZ`	Optional end time (naive, no tz conversion)
`description`	`TEXT`	LLM-generated summary or user-provided notes
`meeting_title`	`TEXT`	Meeting title
`billable`	`BOOLEAN`	Whether the entry is billable
`confidence`	`NUMERIC`	Confidence score (`0.75` for KB-pushed; `0.0` for manual)
`status`	`TEXT`	`logged` (default)
`organizer`	`TEXT`	Meeting organizer
`attendees`	`TEXT`	Comma-separated attendee list

`chat_feedback` table (TTT DB)

Stores chat response ratings for the feedback loop. Created via `init_db()` using `CREATE TABLE IF NOT EXISTS`.

Column	Type	Description
`id`	`VARCHAR(36)` PK	UUID string
`user_id`	`VARCHAR(36)`	Supabase user UUID
`question`	`TEXT`	The user's question
`answer`	`TEXT`	The LLM-generated answer that was rated
`sources`	`JSONB`	Array of source objects `{source, score, chunk_index}` used in the answer

Column	Type	Description
`rating`	`SMALLINT`	`1` = thumbs up, `-1` = thumbs down
`note`	`TEXT`	Optional free-text note (reserved for future UI)
`created_at`	`TIMESTAMPTZ`	Timestamp of rating

An index on `(user_id)` makes the stats query fast.

---

3. Configuration layer

File: `backend/kb/config.py`

All configuration is environment-driven. `config.py` calls `load_dotenv()` against candidate paths in priority order:

1. `KB_ENV_FILE` env var (explicit override)
2. `./env` in the current working directory
3. `.env` in the project root

Every other module imports constants directly from `config.py`. Nothing reads `os.environ` elsewhere.

Environment variable reference

Variable	Description
`DATABASE_URL`	pgvector PostgreSQL connection string
`TTT_DATABASE_URL`	Separate TTT PostgreSQL connection string
`SUPABASE_URL`	Supabase project URL (used to fetch JWKS)
`SUPABASE_JWT_SECRET`	Static JWT secret (HS256 fallback)
`EMBED_PROVIDER`	`nomic` or `ollama`
`NOMIC_API_KEY`	Required when `EMBED_PROVIDER=nomic`
`EMBED_DIMENSIONS`	Vector dimension — must match the embedding model (default `768`)
`LLM_PROVIDER`	`ollama`, `openai`, or `watsonx`
`OLLAMA_HOST`	Ollama base URL (default `http://localhost:11434`)
`OLLAMA_CHAT_MODEL`	Ollama chat model name
`OLLAMA_EMBED_MODEL`	Ollama embedding model name
`OPENAI_API_KEY`	API key for any OpenAI-spec provider
`OPENAI_BASE_URL`	Base URL override (e.g. Groq, ZhipuAI)
`OPENAI_CHAT_MODEL`	Model name
`WATSONX_API_KEY`	IBM Cloud API key

Variable	Description
`WATSONX_PROJECT_ID`	watsonx.ai project ID
`WATSONX_MODEL_ID`	watsonx.ai model ID
`WATSONX_URL`	watsonx.ai inference URL
`RAG_TOP_K`	Default number of chunks retrieved per search
`USE_LANGCHAIN`	`true` to route RAG + agentic calls through LangChain; `false` (default) uses custom pipeline

Environment variable precedence (OpenShift)

```
OpenShift Secret (via deploy.sh)       ← highest
  ↓
deploy.env / render.yaml envVars
  ↓
.env file (local only)
  ↓
config.py defaults                     ← lowest
```

---

4. Authentication

File: `backend/kb/auth.py`

All routes that read or write user data use `get_current_user` as a FastAPI dependency. It validates the Supabase JWT from the `Authorization: Bearer` header and returns the user's UUID (`sub` claim).

Token validation

Supabase projects may sign JWTs with either HS256 (static secret, older projects) or ES256 (rotating asymmetric key from JWKS, newer projects). The validator handles both:

1. **ES256 via JWKS** — on first call, fetches `{SUPABASE_URL}/auth/v1/.well-known/jwks.json` and caches key objects by `kid`. Each incoming token's `kid` is looked up in the cache; if not found the JWKS is re-fetched once before failing over to HS256.
2. **HS256 static secret** — falls back to `SUPABASE_JWT_SECRET` if JWKS is unavailable or the token's `kid` is not found.

The JWKS key cache is protected by a `threading.Lock` — safe for concurrent requests.

User isolation

Every database query in the ingestion, search, chat, and TTT paths includes a `WHERE user_id = :user_id` clause (or equivalent ORM filter) using the UUID returned by `get_current_user`. Users cannot read or modify other users' documents or time entries.

---

5. Ingestion pipeline

Files: `backend/kb/ingest.py` · `backend/kb/extractors.py`

Entry points:

- **API:** `POST /upload` (multipart) and `POST /ingest-meeting` (multipart)
- **CLI:** `kb ingest <path>`

Both converge on `ingest(path, force=False, source_name=None, user_id=None)`.

Step 1 — File discovery

`_iter_files(root)` walks the path. A single file is yielded directly. A directory recursively yields all files whose suffix is in `SUPPORTED_SUFFIXES`.

Step 2 — Dedup check

`_already_indexed(session, source_key, user_id)` checks for an existing row with the same `source` and `user_id`. If found and `force=False`, the file is skipped. If `force=True`, all existing rows for that source and user are deleted before re-ingesting. The `source_key` is the original upload filename — the DB key is always the human-readable name, not a temp path.

Step 3 — Extraction and chunking

`extract(path)` in `extractors.py` dispatches by file extension:

Suffix	Extractor	Notes
`.pdf`	`extract_pdf`	pypdf — concatenates all page text
`.docx`, `.doc`	`extract_docx`	python-docx — joins paragraph text
`.png`, `.jpg`, `.jpeg`, `.gif`, `.bmp`, `.tiff`, `.webp`	`extract_image`	Pillow + pytesseract OCR
`.txt`, `.md`, `.csv`, `.json`, `.yaml`, `.html`, `.py`, `.js`, `.ts`, `.go`, etc.	`extract_txt`	Plain read + meeting metadata extraction
anything else	`extract_generic`	Strips non-ASCII bytes, extracts printable characters

All extractors funnel through `_chunk_text(text, size=800, overlap=100)` — fixed-size character chunks with 100-character overlap to preserve context at boundaries.

Meeting metadata extraction

`extract_txt` also calls `extract_meeting_metadata(raw)` which scans for structured header fields:

Header	Stored as
`Date: …`	`meeting_date` (normalised to ISO-8601)

Header	Stored as
`Meeting Title: …`	`meeting_title`
`Time: …`	`meeting_time` (raw string, e.g. `2:00 PM – 2:45 PM CST`)
`Organizer: …`	`organizer`
`Attendees: …`	`attendees`
`Platform: …`	`platform`
`Project: …` / `Project Code: …`	`project_code`
`Billable: …`	`billable` (normalised to `bool`)
`Duration: …` / `Meeting Duration: …`	`duration_minutes` (normalised to `float`)

Each chunk is prefixed with a short `[Meeting: … | Date: … | Organizer: …]` header so every chunk carries context about its source meeting even when retrieved in isolation.

Step 4 — Embedding

`embed(chunk)` is called synchronously for each chunk, returning a `list[float]` of length `EMBED_DIMENSIONS` (768).

Step 5 — Database write

A `Document` ORM row (with `user_id`) is added to the SQLAlchemy session for each chunk. The session commits after all chunks of a single file are processed. A failure rolls back only that file.

---

6. Embedding layer

File: `backend/kb/embedder.py`

```
embed(text: str) -> list[float]
embed_batch(texts: list[str]) -> list[list[float]]
```

Both delegate to the provider returned by `get_embedder()` which reads `EMBED_PROVIDER`.

OllamaEmbedder

Calls Ollama's `/api/embed` via `httpx` with `Connection: close` to force HTTP/1.1 and avoid connection-reuse issues. Returns `embeddings[0]`.

NomicEmbedder

Calls the Nomic Atlas API at `https://api-atlas.nomic.ai/v1/embedding/text` with `task_type: "search_document"`. Both providers output 768-dimensional vectors matching the `Vector(768)`

pgvector column.

---

7. Semantic search

File: `backend/kb/search.py`

`search(query, top_k, file_type, source_filter, user_id)`

1. Embeds the query with `embed(query)`
2. Runs a SQLAlchemy query using pgvector's cosine distance operator `<=>`:

```
SELECT *, embedding <=> :query_vector AS distance
FROM documents
WHERE user_id = :user_id
[AND file_type = :file_type]
[AND source ILIKE '%source_filter%']
ORDER BY distance
LIMIT :top_k
```

3. Converts distance to similarity score: `score = 1.0 - distance`
4. For each result, calls `_extract_snippet(content, query)` for a short excerpt centred on the best-matching region

Snippet extraction

`_extract_snippet` uses a sliding window algorithm:

- Tokenises the query into non-trivial terms (≥3 chars, minus stop words)
- Finds all character positions where each term appears in the chunk
- Slides over match positions to find the window covering the most distinct query terms
- Trims to sentence or word boundaries and adds ellipsis markers

Supporting queries

Function	Purpose
`get_most_recent_meeting_date(user_id)`	Returns the latest `doc_metadata->>'meeting_date'` for temporal re-ranking in chat
`list_sources(user_id)`	`GROUP BY source, file_type` with chunk count
`delete_source(source, user_id)`	Deletes all rows matching `source` and `user_id`

---

8. RAG chat pipeline

File: `backend/kb/chat.py`

Entry point: `ask(question, history, top_k, source_filter, file_type, skip_ttt, user_id)`

Step 1 — Intent classification (pre-LLM, regex)

`_is_temporal_meeting_query(question)` — matches "last meeting", "most recent standup", "latest sync", etc. When true, triggers meeting date re-ranking and forces TTT meeting history lookup.

`is_ttt_query(question)` (from `ttt.py`) — matches "hours logged", "billable", "time entries", "what did I work on", etc. When true, fetches structured TTT rows.

Step 2 — Vector retrieval

Calls `search(question, top_k, file_type, source_filter, user_id)`. Returns up to `top_k` `SearchResult` objects ordered by cosine similarity.

Step 3 — Temporal meeting re-ranking

If `_is_temporal_meeting_query` fired:

1. `get_most_recent_meeting_date(user_id)` fetches the latest `meeting_date` in `doc_metadata`
2. Retrieved chunks are partitioned: chunks from that date first, all others after
3. If no chunks from the most recent date are in the top-K, a second search runs with `top_k * 2` and the same partitioning applies

Step 4 — TTT context injection

If `skip_ttt=False` and TTT or temporal intent was detected, `query_ttt(question, user_id)` fetches structured rows from the TTT database and prepends them as a text block to the vector context — structured data before unstructured prose. `skip_ttt=True` is passed from `/summarize-meeting` to prevent historical entries from contaminating the fresh meeting summary.

Step 5 — Prompt construction

```
[system]
You are a helpful assistant with access to a personal knowledge base and a
Time Task Tracker (TTT) database of logged work entries.
Answer the user's question using ONLY the context passages provided below.
...

Context:
{ttt_rows}

[1] filename.txt (chunk 2, date 2026-08-16, score 0.821):
<chunk text>

[2] …

[user turn N-2] …
```

```
[assistant turn N-1] …
[user] <current question>
```

Conversation history is appended as alternating `user`/`assistant` messages for multi-turn context. Retrieval always uses only the latest question so search stays focused.

Step 6 — LLM call

`get_provider().chat(messages)` dispatches to the configured provider. Temperature 0 is used for deterministic grounded answers.

Step 7 — Response

Returns `ChatResponse(answer: str, sources: list[dict])` where each source contains `source`, `score`, and `chunk_index`.

---

9. LLM provider abstraction

File: `backend/kb/llm.py`

All providers implement `BaseLLMProvider.chat(messages: list[dict]) -> str`. Messages follow the OpenAI format: `[{"role": "system"|"user"|"assistant", "content": "…"}]`.

OllamaProvider

Posts to `{OLLAMA_HOST}/api/chat` with `stream=False`. Returns `response["message"]["content"]`.

OpenAIProvider

Posts to `{OPENAI_BASE_URL}/chat/completions`. Compatible with any OpenAI-spec endpoint — OpenAI, Groq, GLM-4 (ZhipuAI), Anyscale, Together, etc. Returns `choices[0].message.content`.

WatsonxProvider

Two-step per call:

1. **IAM token exchange** — `POST https://iam.cloud.ibm.com/identity/token` with `grant_type=apikey` → `access_token`
2. **Inference** — `POST {WATSONX_URL}/ml/v1/text/chat?version=2023-05-29` with token in `Authorization` header

Parameters: `temperature=0`, `max_tokens=600`, `frequency_penalty=0`, `presence_penalty=0`, `top_p=1`.

> **Note:** The IAM token is fetched fresh on every call (no cache). Each call adds ~200–400 ms for the token exchange round-trip.

Adding a new provider

1. Subclass `BaseLLMProvider` in `llm.py` and implement `chat()`
2. Register it in `get_provider()` with a string key
3. Set `LLM_PROVIDER=<key>` in `.env`

---

10. Meeting intelligence pipeline

Files: `backend/api.py` · `backend/kb/extractors.py` · `backend/kb/pusher.py`

This pipeline is split across two HTTP requests to avoid the 30-second response timeout (the LLM call alone takes 15–25 s).

Request 1 — `POST /ingest-meeting`

1. Saves the uploaded file to a temp path
2. Calls `ingest(tmp_path, force=force, source_name=file.filename, user_id=user_id)`
3. Deletes the temp file
4. Returns `{status: "ok", filename: "…"}` immediately (seconds)

The file is now in pgvector.

Request 2 — `POST /summarize-meeting`

Receives `{filename, project_code, organizer, attendees}` as JSON.

1. **Metadata lookup** — calls `kb_search(filename, top_k=1, source_filter=filename, user_id=user_id)` to extract `doc_metadata`
2. **RAG summarization** — calls `kb_ask(summary_question, source_filter=filename, top_k=3, skip_ttt=True, user_id=user_id)`. `source_filter` scopes retrieval to only this file's chunks
3. **Time parsing** — `_parse_time_range(meeting_time, entry_date)` in `pusher.py` parses the raw time string into two naive `datetime` objects
4. **TTT push** — `push_meeting_entry(...)` inserts into `time_entries`
5. Returns `IngestMeetingResponse` with LLM summary, source citations, TTT entry ID, and any push error

Field sourcing priority

TTT field	Source priority
`meeting_title`	`doc_metadata.meeting_title` → filename
`entry_date`	`doc_metadata.meeting_date` → today
`start_time` / `end_time`	parsed from `doc_metadata.meeting_time` → null
`duration_minutes`	`doc_metadata.duration_minutes` → parsed from LLM summary → 60
`project_code`	request body → `doc_metadata.project_code` → parsed from LLM summary → filename stem

TTT field	Source priority
`organizer`	request body → `doc_metadata.organizer` → null
`attendees`	request body → `doc_metadata.attendees` → null
`billable`	`doc_metadata.billable` → false
`description`	LLM-generated summary

11. Agentic meeting summariser

File: `backend/api.py` — `POST /agentic-meeting`

After a transcript has been ingested (via `POST /ingest-meeting`), the agentic endpoint runs a multi-step tool-call pipeline. The implementation used depends on `USE_LANGCHAIN`.

Custom pipeline (`USE_LANGCHAIN=false`, default)

Deterministic 5-step sequence — all steps always run in order:

Step	Tool name	Implementation	What it does
1	`search_kb`	`kb_search(query, source_filter=filename)`	Retrieves the top-5 transcript chunks
2	`lookup_ttt`	`query_ttt(f"recent meetings for project {project}", force_meetings=True)`	Fetches past meeting entries for historical context
3	`classify`	`_classify(filename, organizer)` (from `ttt_api`)	Infers project code, task type, billable flag
4	`synthesise`	`get_provider().chat(messages)`	LLM call with all gathered context (chunks + TTT history)
5	`push_ttt`	`push_meeting_entry(…)`	Inserts the time entry into `time_entries`

LangChain pipeline (`USE_LANGCHAIN=true`)

File: `backend/kb/lc_agent.py`

Uses a LangChain `AgentExecutor` with three `@tool`-decorated functions. The LLM dynamically decides which tools to call, in what order, and whether to retry:

Tool	What it does
`search_kb`	Calls `kb_search()` — same underlying pgvector query
`lookup_ttt`	Calls `query_ttt()` — same TTT SQL query

Tool	What it does
`push_to_ttt`	Calls `push_meeting_entry()` — same DB insert

The `classify` step is removed — the LLM infers the project code from context when constructing the `push_to_ttt` call. The agent may call tools multiple times or in a different order if it determines more context is needed (up to `max_iterations=8`).

Every step's input and output is recorded as an `AgentStep` and returned in the response. The frontend renders these as a collapsible **Agent trace** panel — identical UI for both implementations.

---

12. LangChain pipeline

Files: `backend/kb/lc_embedder.py` · `backend/kb/lc_llm.py` · `backend/kb/lc_chat.py` · `backend/kb/lc_agent.py`

Activated when `USE_LANGCHAIN=true`. All four files are lazy-imported inside `chat.py:ask()` and `api.py:agentic_meeting()` — the LangChain packages are not loaded at all when the flag is `false`.

lc_embedder.py

Wraps the existing `NomicEmbedder` and `OllamaEmbedder` classes in LangChain's `Embeddings` interface (`embed_documents` / `embed_query`). The underlying HTTP calls are identical — this is purely an adapter so LC chains can use them.

lc_llm.py

`get_lc_llm()` factory — returns `ChatWatsonx` / `ChatOpenAI` / `ChatOllama` based on `LLM_PROVIDER`. Parameters (model ID, API key, URL) are read from the same `config.py` constants used by the custom providers.

lc_chat.py

LCEL RAG chain — drop-in for `kb/chat.py`. Retrieval is identical (same `search()` and `query_ttt()` calls). The LLM call changes:

```
Custom:   llm.chat([{"role": "system", ...}, ...history..., {"role":
"user", ...}])
LC:       (ChatPromptTemplate | ChatWatsonx |
StrOutputParser).invoke({...})
```

History is converted from `ChatMessage` dataclass objects to LangChain `HumanMessage`/`AIMessage` and passed into a `MessagesPlaceholder` in the prompt template.

lc_agent.py

`run_agentic_meeting()` — replaces the fixed 5-step sequence in `api.py`. Uses `create_tool_calling_agent` + `AgentExecutor`. Tool functions use `threading.local()` to access

request-scoped values (`user_id`, `filename`) without global state, making concurrent requests safe.

Switching between implementations

```
# Enable LangChain
USE_LANGCHAIN=true   # in deploy.env, then re-run deploy.sh

# Revert to custom
USE_LANGCHAIN=false
```

No rebuild required — the flag is read from the environment at runtime on each request.

---

13. Chat feedback loop

Files: `backend/api.py` · `backend/kb/db.py` · `frontend/src/components/Chat.jsx` · `frontend/src/components/FeedbackStats.jsx`

Write path (`POST /feedback`)

Each assistant message in the Chat UI carries 👍/👎 buttons. On click, the frontend calls `POST /feedback` with:

- `question` — the user's query
- `answer` — the full LLM response
- `sources` — the retrieved source list (for context on why the answer may have been poor)
- `rating` — `1` or `-1`

The backend inserts a row into `chat_feedback` using the user's `user_id` from the JWT. Once rated, buttons are replaced with a confirmation message and cannot be re-clicked for the same message (session-local state).

Read path (`GET /feedback/stats`)

Returns aggregated statistics for the Feedback tab:

```
{
  "total": 42,
  "thumbsUp": 35,
  "thumbsDown": 7,
  "score": 83.3,
  "lowRated": [
    {
      "id": "…",
      "question": "What did we decide about the pricing model?",
      "answer": "I don't have that information.",
      "sources": [],
      "note": null,
      "createdAt": "2025-07-14T10:22:00+00:00"
```

```
      }
    ]
  }
```

Why this teaches RLHF patterns

The `chat_feedback` table is the data collection layer of a human feedback loop. The low-rated query log in the Feedback tab surfaces exactly what a real RLHF workflow uses for annotation: the original question, the model's answer, and the retrieved context that was available. These can be used to:

- Identify knowledge gaps (questions with no relevant chunks → upload missing docs)
- Tune the system prompt (consistently wrong answer style)
- Improve retrieval (relevant chunks not retrieved → adjust chunk size or top-K)

---

14. Time Task Tracker — write path

File: `backend/kb/pusher.py`

`push_meeting_entry()` uses a direct psycopg2 connection to the TTT database. The `INSERT … ON CONFLICT (id) DO NOTHING` pattern makes duplicate pushes idempotent — the same UUID will not overwrite an existing entry.

Time range parsing

`_parse_time_range(time_str, entry_date)` handles formats including:

- `10:00 AM – 10:30 AM CST`
- `14:00 – 14:45`
- `2:00 PM – 2:45 PM`

The regex `(\d{1,2}:\d{2})\s*(AM|PM)?` finds all time tokens. The first becomes `start_time`, the second becomes `end_time`. Both are constructed as naive `datetime` objects (no `tzinfo`) so the TTT displays wall-clock time as written.

Duration parsing

`_parse_duration_minutes(text)` applies two regexes to the LLM summary or the raw header value:

- `(\d+)\s*(?:hour|hr|h)\b` → multiply by 60
- `(\d+)\s*(?:minute|min|m)\b` → add directly

Defaults to 60 minutes if nothing is found.

---

15. Time Task Tracker — read path (chat context)

File: `backend/kb/ttt.py`

`query_ttt(question, user_id, limit, force_meetings)` is called from `chat.py` when TTT intent is detected. It classifies the question into one of four SQL shapes:

Shape	Trigger	Query
Meeting list	`force_meetings=True` or meeting history pattern	`SELECT … WHERE user_id=? AND task_type='meeting' ORDER BY entry_date DESC`
Aggregated totals	"total", "sum", "how many hours"	`SELECT SUM(duration_minutes), COUNT(*) GROUP BY project_code, task_type`
Billable filter	"billable"	`SELECT … WHERE user_id=? AND billable = TRUE`
Default recent	fallback	`SELECT … WHERE user_id=? ORDER BY entry_date DESC`

All shapes support:

- **Date range** — extracted from "today", "this week", "last month", etc. Defaults to ±365 days (wide window to handle future-dated entries such as upcoming meetings)
- **Project filter** — `ILIKE '%project%'` extracted from "for Honda", "on Honda", "Honda meeting"
- **Count** — extracted from "last two meetings", "last 3 entries" — overrides the default `limit`

Results are formatted as a readable text block prepended with `[Time Task Tracker — N result(s), date to date]` and injected ahead of vector chunks in the RAG prompt.

---

16. TTT REST API

File: `backend/ttt_api.py`

Mounted at `/ttt`. All routes require a valid Supabase JWT and scope every query to `user_id`.

Route	Method	Description
`/ttt/entries`	`GET`	List entries with optional `start_date`, `end_date`, `project_code` filters
`/ttt/entries`	`POST`	Create a new entry (returns 201)
`/ttt/entries/bulk-delete`	`POST`	Delete a list of entries by ID
`/ttt/entries/{id}`	`GET`	Fetch a single entry
`/ttt/entries/{id}`	`PUT`	Update an entry (partial update)
`/ttt/entries/{id}`	`DELETE`	Delete an entry (204)
`/ttt/summary`	`GET`	Aggregated totals grouped by project and task type for a date range
`/ttt/export/csv`	`GET`	Download entries as CSV for a date range

Route	Method	Description
`/ttt/import/csv`	`POST`	Bulk-import entries from a CSV file
`/ttt/import/ics`	`POST`	Bulk-import entries from an ICS calendar file
`/ttt/projects`	`GET`	List all distinct project codes for the user
`/ttt/classify`	`POST`	Infer project code and billable flag from a meeting title

KB routes (mounted on app root in `api.py`)

Route	Method	Description
`/upload`	`POST`	Ingest a file into pgvector
`/search`	`POST`	Semantic search
`/sources`	`GET`	List indexed sources
`/sources`	`DELETE`	Delete a source
`/chat`	`POST`	RAG chatbot
`/ingest-meeting`	`POST`	Ingest meeting transcript (step 1)
`/summarize-meeting`	`POST`	Standard RAG summarise + push to TTT (step 2)
`/agentic-meeting`	`POST`	Agentic 5-step summarise + push to TTT (step 2 alternative)
`/feedback`	`POST`	Store a chat response rating (1 or -1)
`/feedback/stats`	`GET`	Approval score, total counts, low-rated query log
`/health`	`GET`	Health check (no auth)

CSV import format

Accepted column names are flexible (case-insensitive, normalised). Minimum required: a date column and a duration column. Optional: `meeting_title`, `project_code`, `task_type`, `billable`, `description`, `organizer`, `attendees`.

ICS import

Parses `VEVENT` blocks. `DTSTART`/`DTEND` provide start time, end time, and date. `SUMMARY` maps to `meeting_title`. Duration is computed from `DTEND - DTSTART`. The LLM `/ttt/classify` endpoint is **not** called during ICS import — project codes default to `GENERAL` and billable defaults to `false` unless the title matches built-in keyword heuristics.

AI classification (`/ttt/classify`)

`_classify(title, organizer)` applies regex patterns to infer:

- **Project code** — keyword patterns extracted from common project name formats

- **Billable** — matched against lists of billable keywords (`client`, `customer`, `consulting`) and non-billable keywords (`internal`, `team`, `admin`, `training`)

---

17. Frontend architecture

Files: `frontend/src/`

Single-page React application built with Vite. All API communication goes through `frontend/src/api.js` (KB + feedback + agentic routes) and `frontend/src/tttApi.js` (TTT routes). Both attach `Authorization: Bearer {token}` to every request.

Auth flow

`frontend/src/context/AuthContext.jsx` wraps the app in a `AuthProvider`. On mount it calls `supabase.auth.getSession()` and then subscribes to `onAuthStateChange` to keep the session in sync across tabs, token refreshes, and logouts. The `useAccessToken()` hook is the standard way to get the current JWT for API calls.

If the session is `null` (logged out), `App.jsx` renders `LoginPage.jsx` instead of the main tabs.

Login page

`LoginPage.jsx` supports:

- **Magic link** — `supabase.auth.signInWithOtp({ email })` — sends a login link to the user's email
- **Google OAuth** — `supabase.auth.signInWithOAuth({ provider: "google" })`
- **GitHub OAuth** — `supabase.auth.signInWithOAuth({ provider: "github" })`

KB components

Component	Responsibility
`Chat.jsx`	Multi-turn conversation UI. Renders user/assistant bubbles, collapsible source citations, thinking state. Each assistant message has 👍/👎 feedback buttons; rating is stored via `POST /feedback` and the button state flips to a confirmation.
`Search.jsx`	Semantic search form with file type filter, source substring filter, top-K control. Snippet cards with expand-to-full-chunk toggle.
`Upload.jsx`	react-dropzone multi-file queue. Per-file upload status, force re-index checkbox.
`MeetingUpload.jsx`	Two-phase upload. **Mode toggle**: Standard RAG (calls `/summarize-meeting`) or Agentic (calls `/agentic-meeting`). In agentic mode, an Agent trace panel renders each tool call with icon, input, and truncated output.
`Sources.jsx`	Table of all indexed sources with chunk counts. Per-row delete with confirmation.

Component	Responsibility
`FeedbackStats.jsx`	Feedback dashboard. Fetches `/feedback/stats`, renders 4 stat tiles (total, 👍, 👎, score %), a proportional progress bar, and an expandable log of every low-rated query with its full question, model answer, and sources.

TTT components

Component	Responsibility
`TTTDashboard.jsx`	Stat cards (total hours, meeting count, billable hours) and bar chart of hours by project for a selected date range
`TTTEntries.jsx`	Filterable, sortable list of time entries. Inline edit, per-row delete, bulk delete with checkbox selection.
`TTTManualEntry.jsx`	Form to create a new entry with project code, task type, date, duration, billable flag, start/end time, and notes
`TTTImport.jsx`	react-dropzone zones for CSV import and ICS calendar import. Shows imported count and failure count.
`TTTReports.jsx`	Date-range report generation (summary grouped by project + task type). CSV export download.

API layer

```
// KB API — api.js
uploadFile(file, force, token)                          // POST /upload
searchDocs({ query, top_k, … }, token)                  // POST /search
chatWithKB({ question, history, … }, token)             // POST /chat
getSources(token)                                       // GET /sources
deleteSource(source, token)                             // DELETE /sources?
source=…
ingestMeeting({ file, force }, token)                   // POST /ingest-
meeting
summarizeMeeting({ filename, … }, token)                // POST /summarize-
meeting
agenticMeeting({ filename, … }, token)                  // POST /agentic-
meeting
submitFeedback({ question, answer, sources, rating }, token) // POST
/feedback
getFeedbackStats(token, limit)                          // GET
/feedback/stats

// TTT API — tttApi.js
getEntries({ startDate, endDate, projectCode }, token)  // GET /ttt/entries
createEntry(entry, token)                               // POST
/ttt/entries
updateEntry(id, updates, token)                         // PUT
/ttt/entries/{id}
deleteEntry(id, token)                                  // DELETE
```

```
  /ttt/entries/{id}
  bulkDeleteEntries(ids, token)                          // POST
  /ttt/entries/bulk-delete
  getSummary({ startDate, endDate }, token)              // GET /ttt/summary
  exportCSV(token, startDate, endDate)                   // GET
  /ttt/export/csv (blob)
  importCSV(file, token)                                 // POST
  /ttt/import/csv
  importICS(file, token)                                 // POST
  /ttt/import/ics
  getProjects(token)                                     // GET
  /ttt/projects
  classifyMeeting(title, organizer, token)               // POST
  /ttt/classify
```

`throwApiError(res)` is a shared helper in both API files that parses FastAPI error responses — tries `res.json().detail` first, falls back to `res.text()`.

---

18. OpenShift deployment

Files: `openshift/`

Key files

File	Purpose
`deploy.sh`	One-command deploy: build multi-arch images (amd64 + arm64), push to registry, log into cluster, apply secrets, deploy pods, wait for `Running`, print URL
`dump.sh`	Dump both databases to `openshift/backup.sql` before cluster expiry
`deploy.env.example`	Template — copy to `deploy.env` and fill in secrets
`postgres.yaml`	pgvector `StatefulSet` + Ceph RBD `PersistentVolumeClaim`
`backend.yaml`	FastAPI `Deployment` + `ClusterIP Service`
`frontend.yaml`	nginx `Deployment` + `Service` + OpenShift `Route` (HTTPS)
`secret.yaml`	Secret template (reference only — do not commit with values)
`Dockerfile.backend`	FastAPI image (built from `./backend`)
`Dockerfile.frontend`	Vite build → nginx image; `VITE_API_URL=/api` baked in
`nginx.conf`	Serves static files on port 8080, proxies `/api/*` to backend ClusterIP

Cluster migration workflow

```
  # Before expiry — dump both DBs
  ./openshift/dump.sh      # writes openshift/backup.sql
```

```
# Update credentials in deploy.env
OC_SERVER=https://api.new-cluster.com:6443
OC_TOKEN=sha256~new-token

# Deploy to new cluster — backup.sql is auto-restored
./openshift/deploy.sh
```

`deploy.sh` detects `openshift/backup.sql` and runs `psql` restore inside the new postgres pod before the backend starts.

Post-deploy step

After each new cluster deploy, update **Supabase → Authentication → URL Configuration**:

- **Site URL** → `https://<route-host>`
- **Redirect URLs** → `https://<route-host>/**`

Without this, login redirects will fail.

---

19. File reference

Configuration

Path	Description
`.env`	Primary local config
`.env.example`	Template for `.env`
`openshift/deploy.env`	OpenShift secrets (gitignored)
`openshift/deploy.env.example`	Template for `deploy.env`
`recorder/.env`	Recorder settings — `KB_INGEST_URL`, `WHISPER_MODEL`, `AUDIO_DEVICE`, `RECORDINGS_DIR`
`frontend/.env.local`	Vite env — `VITE_API_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`

Core library — `backend/kb/`

File	Description
`config.py`	Loads `.env`, exports all config constants (incl. `USE_LANGCHAIN`)
`auth.py`	Supabase JWT validation — ES256 (JWKS) + HS256 (static secret), returns `user_id`
`db.py`	SQLAlchemy engine, `SessionLocal`, `Document` ORM model, `init_db()`
`embedder.py`	`embed(text) → list[float]` — Nomic or Ollama provider

File	Description
`extractors.py`	File-type dispatch → 800-char / 100-overlap chunking + meeting metadata extraction
`ingest.py`	Orchestrates discovery → dedup → extract → embed → upsert, with user isolation
`search.py`	Cosine search, snippet extraction, `list_sources`, `delete_source`
`chat.py`	RAG pipeline: intent classify → retrieve → re-rank → TTT inject → prompt → LLM; routes to `lc_chat.py` when `USE_LANGCHAIN=true`
`llm.py`	`BaseLLMProvider` ABC → `OllamaProvider` / `OpenAIProvider` / `WatsonxProvider`
`lc_embedder.py`	LangChain `Embeddings` adapter wrapping `NomicEmbedder` / `OllamaEmbedder`
`lc_llm.py`	`get_lc_llm()` factory → `ChatWatsonx` / `ChatOpenAI` / `ChatOllama`
`lc_chat.py`	LCEL RAG chain — drop-in for `chat.py` when `USE_LANGCHAIN=true`
`lc_agent.py`	LangChain `AgentExecutor` — drop-in for the fixed 5-step agentic pipeline
`pusher.py`	Writes meeting summaries to `time_entries` via direct psycopg2
`ttt.py`	Reads TTT rows for RAG context injection
`cli.py`	Click CLI: `kb init` · `kb ingest <path>` · `kb search <query>` · `kb list` · `kb delete <source>`

Backend

File	Description
`backend/api.py`	FastAPI app — KB routes: `/upload`, `/search`, `/sources`, `/chat`, `/ingest-meeting`, `/summarize-meeting`, `/agentic-meeting`, `/feedback`, `/feedback/stats`
`backend/ttt_api.py`	TTT router mounted at `/ttt` — full CRUD, import, export, summary, classify

Frontend

File	Description
`frontend/src/App.jsx`	Root component — login gate, 6 KB tabs (Chat, Search, Upload, Meeting, Sources, Feedback) + 5 TTT tabs
`frontend/src/api.js`	KB fetch wrappers
`frontend/src/tttApi.js`	TTT fetch wrappers
`frontend/src/supabaseClient.js`	Supabase JS client initialised from `VITE_SUPABASE_URL` +

File	Description
	`VITE_SUPABASE_ANON_KEY`
`frontend/src/context/AuthContext.jsx`	Session provider, `useSession()`, `useAccessToken()`
`frontend/src/components/LoginPage.jsx`	Magic link + Google + GitHub sign-in

Recorder (optional, macOS only)

File	Description
`recorder/teams_recorder.py`	ffmpeg + BlackHole audio capture → Whisper transcription → POST /upload

Output saved to `~/recordings/` as `teams_YYYYMMDD_HHMMSS.wav` + `.txt`.